Module Code & Module Title

CC6012NT – Data and Web Development

40% Individual Coursework

Semester

2024/2025 Spring/Autumn

Student Name: Puja Chaudhary

London Met ID: 23049324

College ID: NP05CP4A230158

Assignment Submission Date: Mar 18, 2026, 1:00 PM

Assignment Due Date: Mar 18, 2026, 2:00 PM

Submitted To: Mr. Binaya Koirala

Git Hub Link: **https://github.com/jaiswalpuza2/KumariCinemas.git**

I confirm that I understand my coursework needs to be submitted online via Google Classroom under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Table of Contents

1 Introduction to Case Study 5

1.1 Problem Statement 5

1.2 Proposed Solution 6

2 Textual Analysis 7

3 Initial ERD 8

4 Normalization 9

4.1 UNF 10

4.2 First Normal Form (1NF) 10

4.3 Second Normal Form (2NF) 11

4.4 Third Normal Form (3NF) 14

5 Integration and Assumption 18

6 Final ERD 20

7 Data Dictionary 21

8 Scripts 24

9 Insert Statement 29

10 Select Statement 30

11 Forms 31

11.1 Dashboard Page 31

11.2 Complex Forms and Queries 32

11.3 Simple Forms 37

12 User Manual 39

13 Testing 44

14 Further Discussion 47

15 Conclusion 48

16 References 49

Table of Figures

Figure 1 Initial ERD 8
Figure 2 Final ERD 20
Figure 3 output 27
Figure 4 output 27
Figure 5 output 28
Figure 6 Insert query 29
Figure 7 Insert Query 29
Figure 8 output 30
Figure 9 selecting query 30
Figure 10 output 31
Figure 11 dashboard page 31
Figure 12 dashboard page 32
Figure 13 Ticket History Page 33
Figure 14 Venue page 34
Figure 15 Performance List 34
Figure 16 login page 37
Figure 17 sign up page 38
Figure 18 login page for administrator 39
Figure 19 movie list 40
Figure 20 show list 40
Figure 21 sign up page for customer 41
Figure 22dashboard page 42
Figure 23 ticket list 43
Figure 24 check if invalid email reject 44
Figure 25 show list 45

Table of Tables

Table 1 user-3 21
Table 2 movie-3 21
Table 3 theater-3 22
Table 4 hall-3 22
Table 5 show-3 22
Table 6 ticket_tyep-3 23
Table 7 ticket-3 23
Table 8 Test Cases 46

1 Introduction to Case Study

The cinema chain belongs to KumariCinemas which has multiple theatres in Nepal. The organization is interested in upgrading their current movie ticketing and management system with a centralized, automated system, which is database based. Today, the organization is facing various threats, which include poor ticket management, lack of communication between the locations of the cinemas, high operations, and poor customer experience. (Statista, 2023)

To counter these factors, KumariCinemas will strive to build an extraordinarily strong system that will be able to manage the movie information, movie theatre locations, halls, movie schedules and the booking of the tickets efficiently. It is believed that the proposed system will lead to better operational efficiency and offer a convenient and smooth experience of watching a movie to the customers.

The Kumari Cinemas Management System is a web-based database app that is created so as to ear monitor the affairs of the Kumari Cinema chain of cinemas in Nepal with specific attention paid to its Kathmandu branch. The main aim of the system is the ease of booking tickets by the system users and at the same time establish good management features on behalf of the system administrators to manage films, theaters, showtimes, and occupancy performance. (Oracle Corporation, 2023)

Key features include:

- User registration and safe log in.
- Movie catalog, Films on demand and Showtime.
- Automated seating reservation and tickets.
- Performance live dashboard of the system.
- Automation of policy of cancelling tickets and updating.

1.1 Problem Statement

The existing management system of cinema is not automated and has no central database. Thus, issues regarding the incorrect seat assignment, ineffective ticketing, processing the records manually, and the lack of real-time data are often produced. These issues cause dissatisfaction among customers and inefficiencies.

The system lacks proper management of:

- There were multiple theatres located in various cities.
- Seating halls of different capacity.
- multiple movie performances.
- Increase or decrease in the price of the tickets depending on public holidays or the release of the new film.
- Auto flake of unpurchased bookings before showtime.

1.2 Proposed Solution

The proposed system is a web-based movie-ticket system which is enabled by relational database. The registered users will have the ability to:

- Check in the movie theater on what is playing and at what time.
- Purchase one or more tickets for a specific performance.
- Buy tickets within the stipulated time.
- Automatic cancellation is done in case the ticket is not bought in a period of one hour before the performance.

The system will also take care of theatre information, hall capacity, shows and price of the tickets. Paid tickets will be calculated as the only occupancy set of seats. Such a centralized system will be able to help KumariCinemas lessen redundancy, make better data quality, and provide better customer care.

2 Textual Analysis

The analysis of texts presents primary actors and relations in the cinema booking scope.

Nouns (Potential Entities):

- User: means customers purchasing tickets and administrators maintaining the system.
- Movie: indicates the movies that are in the theater and important details like the title, genre and duration of the movie.
- Theater/Hall represents the actual screening rooms in the cinema.
- Showtime: is a certain expression pertaining to a movie that is displayed in a theater at certain times.
- Ticket: a booking query which connects a user, showtime and a particular seat.

Verbs (Potential Operations):

- Register/Login: Users may create the accounts and authentication.
- Manage: Administrators may add or update movie and theater details.
- Schedule: links movies to theaters and ShowTimes.
- Book: Users select a showtime and seat to generate a ticket.
- Report: The system will produce such performance measures as occupancy rates.

3 Initial ERD

The first Entity Relationship Diagram (ERD) was designed with respect to the business needs and the KumariCinemas case study. It provides a conceptual representation to the movie ticketing system, including the key entities, such as User, MovieShowHall and Ticket and indicates their essential attributes and relationships. This ERD forms the basis of the further optimization of the entity with the help of the normalization process.

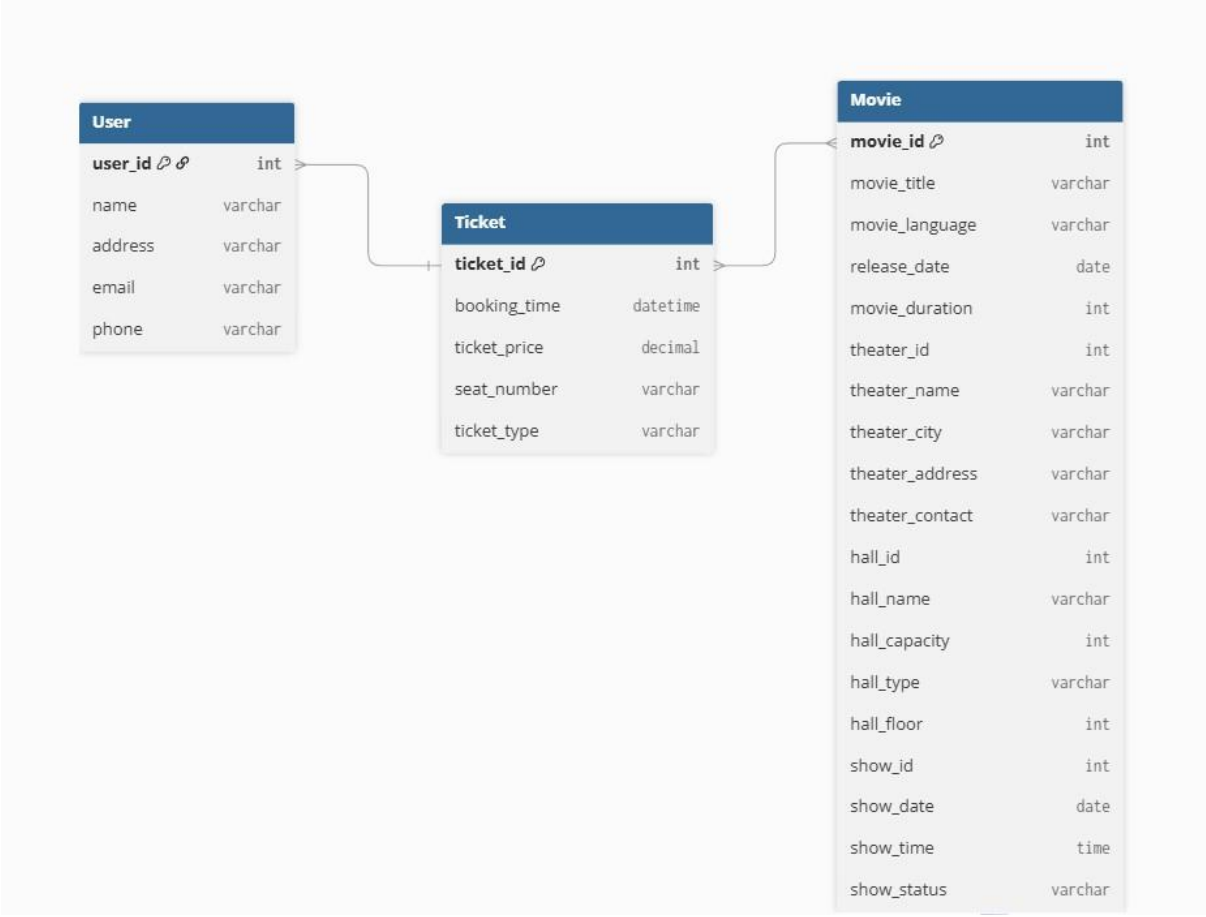

Figure 1 Initial ERD

4 Normalization

Normalization is a particularly important method of database design which aids us to maintain order and efficiency of the data. It breaks the huge unorganized tables into small orderly tables and maintains the relationships intact. (Elmasri, 2016)

The major objectives of normalization are:

- Minimize Data Redundancy: With normalization, redundant information is eliminated, that is, space is saved and a database is made lean.
- Enhance Data Integrity: It eradicates the deletion, insertion, and update anomaly maintaining the data consistency and accuracy.
- Increase Data Maintainability It is easy to make tweaks to the database as businesses change, with a normalized database.
- Make Data Retrieval easy: Structured tabular relationships will enable users to run queries and reports without complexity.

Steps of Normalization:

- Step 1: Unnormalized Form (UNF)

  Each is referred to as a normal form. Based on where in the problem the issue begins, it goes from Unnormalized Form (UNF) with repeating groups or arrays of data to 1NF where values are atomic.
- Step 2: First Normal Form (1NF)

  Each attribute under a pen name has single-valued values as there are no repeating groups and arrays.
- Step 3: Second Normal Form (2NF)

  Deals with cases where there is partial dependency in a table that is a non-key attribute depend on the whole of primary key
- Step 4: Third Normal Form (3NF)

  Third Normal Form eliminates transitive dependencies which are non-key attributes as it depends on the primary key.

4.1 UNF

Unnormalized Form (UNF) is simply the raw, unsanitary information prior to commencement of tidying affairs. Here is where you would find repeating groups and multiple entities being smashed up into one table. In the case of the KumariCinemas movie ticketing system, the raw UNF table would appear like the one below:

user (user_id, name, address, email, phone, ticket_id, booking_time, ticket_price, seat_number, ticket_type {movie_id, movie_title, movie_language, release_date, movie_duration {threater_id threater_name, threater_city, threater_address, threater_contact {hall_id, hall_name, hall_capacity, hall_type, hall_floor}} {show_id, show_date, show_time, show_status}})

4.2 First Normal Form (1NF)

A relational database can be said to meet First normal form (1NF) when all repeating groups are removed, and all attributes hold only atomic (indivisible) values. At this phase, the abnormalized structure is broken down into another tables that present several types of information. Such methodological fragmentation advances redundancy and ensures that single unique values are represented by each field.

In order to achieve 1NF, the nested and repeated data found in the abnormalized stage are isolated into different tables and inter-relationships between the tables conducted via foreign keys. In database normalization, a table is considered to be in First normal form (1NF) when the table contains the following properties:

- Atomic Values: The values of every attribute are to be indivisible and one-valued.
- No Repeating Groups: The repeating groups should be eliminated and separate tables created.

Final 1NF tables:

user-1 (user_id, name, address, email, phone)

movie-1 (movie_id, movie_title, movie_language, release_date, movie_duration)

theatre-1 (threater_id, threater_name, threater_city, threater_address, threater_contact)

hall-1 (hall_id, hall_name, hall_capacity, hall_type, hall_floor, threater_id*)

show-1 (show_id, show_date, show_time, show_status, movie_id*, hall_id*)

ticket-1 (ticket_id, booking_time, ticket_price, seat_number, ticket_type, user_id*, show_id*)

4.3 Second Normal Form (2NF)

The Second Normal Form (2NF), in simple terms, follows the first stage of database normalization the First Normal Form (1NF). In 2NF partial dependencies are eliminated so that any non-key fields depend on all of the fields in the primary key that is data are split into several tables such that each table contains information about a single entity and attributes are placed in the most appropriate table example student data are stored in one table separate from module data and a mapping between modules and teachers are stored in another table.

Key Points:

1. Partial Dependency A partial dependency occurs when non-key attribute depends on a part and not the entirety of the primary key. This kind of state may trigger redundancy and other anomalies in the relational schema.
2. Composite Primary Key: The second normal form(2NF) obliges those tables that use composite primary keys, which are represented by two or more attributes.
3. Conditions for 2NF:

- Must be in 1NF: This table has to be a table that fulfills already the condition of First Normal Form.
- No Partial Dependencies: No non-key attribute is to be functionally dependent on a section of the primary key only.

user-1

user-1 (user_id, name, address, email, phone)
FFD: user_id ⟶ username, address, email, phone,
So: user-2 **(**user_id, name, address, email, phone)

movie-1

movie-1 (movie_id, movie_title, movie_language, release_date, movie_duration)
FFD: movie_id ⟶ movie_title, movie_language, movie_duration, release_date
Therefore,

So: movie-2 (movie_id, movie_title, movie_language, release_date, movie_duration)

theater-1

theatre-1 (threater_id, threater_name, threater_city, threater_address, threater_contact)
Here, theater_id is the primary key.
All non-key attributes are fully dependent on theater_id.
FFD:

theater_id → theater_name, city, threater_address, contact_number

So: theatre-2 (threater_id, threater_name, threater_city, threater_address, threater_contact)

show-1

show-1(show_id, show_date, show_time, show_status, movie_id, hall_id)
FFD:

FFD: show_id → show_date, show_time, show_status, movie_id, hall_id
hence,
show-2 (show_id, show_date, show_time, show_status, movie_id*, hall_id*)

hall-1

hall-1 (hall_id, hall_number, hall_capacity, hall_type, screen_type, theater_id*)

The entity hall-1 contains a single primary key hall_id.

There is no composite key; therefore, partial dependency does not exist.

FFD:

hall_id → hall_number, hall_capacity, hall_type, screen_type, theater_id
Hence, the table formed is:
hall-2 (hall_id, hall_number, hall_capacity, hall_type, screen_type, theater_id*)

In show-1

show-1(show_id, show_time, show_date, ticket_price, show_type, movie_id*, hall_id*)

The entity show-1 has a single primary key show_id.

All non-key attributes are fully dependent on show_id.

FFD: show_id → show_time, show_date, ticket_price, show_type, movie_id, hall_id
Hence, the table formed is
show-2 (show_id, show_time, show_date, ticket_price, show_type, movie_id*, hall_id*)
In ticket-1

ticket-1 (ticket_id, seat_number, booking_status, payment_status, booking_time, user_id*, show_id*)

The entity ticket-1 has a single primary key ticket_id.

All non-key attributes depend fully on ticket_id.

FFD: ticket_id → seat_number, booking_status, payment_status, booking_time, user_id, show_id

Hence, the table formed is:

ticket-2 (ticket_id, seat_number, booking_status, payment_status, booking_time, user_id*, show_id*)

Final 2NF tables

user-2 (user_id, name, address, email, phone)

movie-2 (movie_id, movie_title, movie_language, release_date, movie_duration)

theatre-2 (threater_id, threater_name, threater_city, threater_address, threater_contact)

hall-2 (hall_id, hall_name, hall_capacity, hall_type, hall_floor, threater_id*)

show-2 (show_id, show_date, show_time, show_status, movie_id*, hall_id*)

ticket-2 (ticket_id, booking_time, ticket_price, seat_number, ticket_type, user_id*, show_id*)

4.4 Third Normal Form (3NF)

3NF in database design is essentially the third step in base design on top of the first two levels (1NF and 2NF) and is all about eliminating those loathsome transitive dependencies so that we can reduce the amount of redundant data in our database and at the same time maintain our integrity at a high level.

A table is said to be in 3NF provided that it satisfies the following conditions:

- It is already 2NF
- It is not transitively dependent, i.e., non-key attribute does not depend on another non-key attribute.

Transitive Dependency.

A transitive dependency is the one which appears when a non-key attribute is dependent on another non-key attribute rather than on a primary key.

In user-3

user-2(user_id, user_name, email, phone_number, address)
Here, user_id is the primary key.
None of the non-key attributes (user_name, email, phone_number, address) can uniquely identify any other non-key attribute.

Hence, no transitive dependency exists.
user-3(user_id, user_name, email, phone_number, address)

In movie-3
movie-2(movie_id, title, duration, language, release_date)
All non-key attributes are directly dependent on movie_id.
There is no dependency among non-key attributes.

Hence, no transitive dependency exists.

movie-3(movie_id, title, duration, language, genre, release_date)

In theater-2

theater-2(theater_id, theater_name, city, location, contact_number)
Here, theater_id uniquely identifies all non-key attributes.
Attributes such as city or location do not uniquely determine any other non-key attribute.

Hence, no transitive dependency exists.

theater-3(theater_id, theater_name, city, location, contact_number)

In hall-2

hall-2(hall_id, hall_number, hall_capacity, hall_type, hall_floor, theater_id*)

Entity hall-2 contains only one primary key and all non-key attributes are directly dependent on it.

There are no non-key attributes determining other non-key attributes.

Hence, no transitive dependency exists.
hall-3(hall_id, hall_number, hall_capacity, hall_type, hall_floor, theater_id*)

In show-2 show-2(show_id, show_time, show_date, ticket_price, show_type, movie_id*, hall_id*)

All non-key attributes are directly dependent on show_id.

No attribute such as show_type or ticket_price determines another non-key attribute.

Hence, no transitive dependency exists.

show-3(show_id, show_time, show_date, ticket_price, show_type, movie_id*, hall_id*)

In ticket-2

ticket-2(ticket_id, seat_number, booking_status, payment_status, booking_time, user_id*, show_id*)

Here, ticket_id is the primary key.

All non-key attributes depend directly on ticket_id, and none depend on each other.

Hence, no transitive dependency exists.

ticket-3(ticket_id, seat_number, booking_status, payment_status, booking_time,

user_id*, show_id*)

Final 3NF table

user-3(user_id, name, address, email, phone)

movie-3(movie_id, movie_title, movie_language, release_date, movie_duration)

theatre-3(threater_id, threater_name, threater_city, threater_address, threater_contact)

hall-3(hall_id, hall_name, hall_capacity, hall_type, hall_floor, threater_id*)

show-3(show_id, show_date, show_time, show_status, movie_id*, hall_id*)

ticket_type-3(ticket_type_id, type_name)
ticket-3(ticket_id, booking_time, ticket_price, seat_number, ticket_type, user_id*, show_id*)

5 Integration and Assumption

Once the normalization stage between UNF and Third normal form (3NF) was completed, all the entities were included in the single relational database schema.

The system at the beginning had several entities including User, Movie, Theatre, Hall, Show, and Ticket stored in a single unstructured table. The entities were normalized to separate the entities into different relations so as to avoid redundancy and dependency problems.

The final integrated entities are:

User

(user_id PK, name, address, email, phone)

Movie

(movie_id PK, movie_title, movie_language, release_date, movie_duration)

Theatre

(theatre_id PK, theatre_name, theatre_city, threater_address, theatre_contact)

Hall

(hall_id PK, hall_name, hall_capacity, hall_type, hall_floor, theatre_id FK)

Show

(show_id PK, show_date, show_time, show_status, movie_id FK, hall_id FK)

Ticket_Type

(ticket_type_id PK, type_name)

Ticket

(ticket_id PK, booking_time, ticket_price, seat_number, ticket_type_id FK, user_id FK, show_id FK)

Assumptions:

- Unique Seats: Each seat number should match a specific showtime; it is implemented on the application level and requires to be inserted before, otherwise it is rejected.
- Cancellation Policy: Tickets can be cancelled, though the respective records are all created to be used as a part of historical reporting in the future.
- No Overlaps: Administrative procedures ensure that there are no occurrences of showtimes being booked on the same theatre.
- Capacity Constraint: A theatre is not allowed to sell additional tickets other than what its limit depicts.

6 Final ERD

The final ERD is constructed out of the completely normalized (3NF) tables of the KumariCinemas movie ticketing system. There is no extraneous data, nothing is duplicate, and the relationships are dealt with correctly using primary and foreign keys. This ERD is the designer database that is the optimum database that we create and execute to make it come true. (Connolly, 2015)

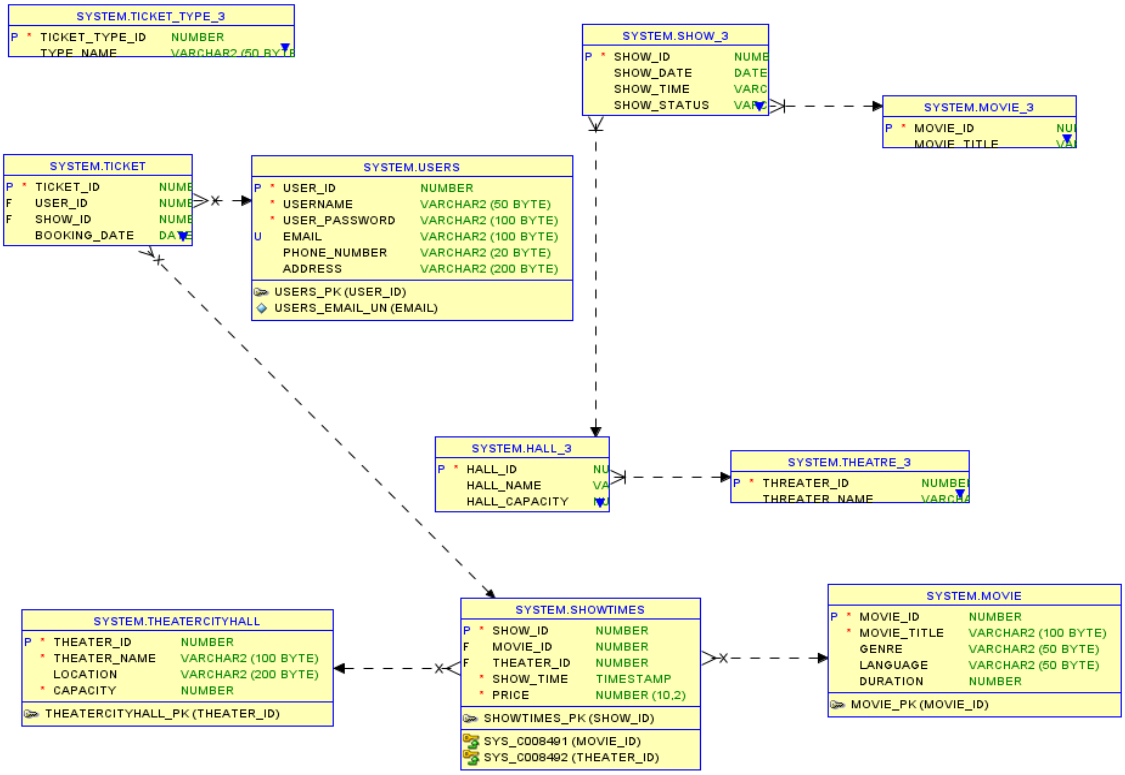

Figure 2 Final ERD

7 Data Dictionary

The Data Dictionary defines each table and each attribute; the types of data, their sizes, constraints, references, and sample values. I extracted it out of the last 3NF ERD.

Table: user-3

Attribute Name	Data type	Size	Key	Description
User-id	Integer		PK	Unique identifier for each user
Name	Varchar	100		Full name of the user
Address	Varchar	255		Residential address of the user
Email	Varchar	100		Email address of the user
phone	Varchar	15		Contact number of the user

Table 1 user-3

` **Table: movie-3**

Attribute name	Data type	Size	Key	Description
Movie_id	Integer		PK	Unique identifier for each movie
Movie_title	Varchar	150		Title of the movie
Movie_language	Varchar	50		Language of the movie
Release_date	Date			Office release date of the movie
Movie_duration	Integer			Duration of the movie in minutes

Table 2 movie-3

Table: threater-3

Attribute name	Data type	Size	Key	Description
Threater_id	Integer		PK	Unique identifier for each threater
Threater_name	Varchar	150		Name of threater
Threater_city	Varchar	100		City where the threater is located

Threater_address	Varchar	255		Physical address of threater
Threater_contact	Varchar	15		Contact number of threater

Table 3 theater-3

Table: hall-3

Attribute name	Data type	Size	Key	Description
Hall_id	Integer		PK	Unique identifier of each hall
Hall_name	Varchar	100		Name of hall
Hall_capacity	Integer			Seating capacity of hall
Hall_type	Varchar	50		Type of hall (standard VIP)
Hall_floor	Integer			Floor number of hall
Threater_id	Integer		FK	

Table 4 hall-3

Table: show-3

Attribute name	Data type	Size	Key	Description
Show_id	Integer		PK	Unique identifier for each show
Show_date	Date			Data of show
show_time	Time			Start time of show
Show_status	Varchar	50		Status of show
Movie_id	Integer		FK	References for the movie being shown
Hall_id	Integer		FK	References for the hall where the show is held

Table 5 show-3

Table: ticket_type-3

Attribute name	Data type	Size	Key	Description
Ticket_type_id	Integer		PK	Unique identifier of ticket type
Type_name	Varchar	50		Name of ticket type either normal, Vip or student

Table 6 ticket_tyep-3

Table: ticket-3

Attribute name	Data type	Size	Key	Description
Ticket_id	Integer		PK	Unique identifier for each ticket
Booking_time	Datetime			Price if ticket
Ticket_price	Decimal			Seat number allocated
Seat_number	Varchar	10		References ticket type
Ticket_type	Integer		FK	References for the user who booked the ticket
User_id	Integer		FK	References for the show for which ticket is booked
Show_id	Integer		FK	

Table 7 ticket-3

8 Scripts

```
CREATE TABLE movie (
   movie_id    NUMBER
      GENERATED ALWAYS AS IDENTITY (START WITH 1 NOCACHE)
   NOT NULL,
   movie_title VARCHAR2(100) NOT NULL,
   genre       VARCHAR2(50),
   language    VARCHAR2(50),
   duration    NUMBER
)
LOGGING;

ALTER TABLE movie ADD CONSTRAINT movie_pk PRIMARY KEY (movie_id);

CREATE TABLE Showtimes (
   show_id    NUMBER
      GENERATED ALWAYS AS IDENTITY (START WITH 1 NOCACHE)
   NOT NULL,
   movie_id   NUMBER,
   theater_id NUMBER,
   show_time TIMESTAMP NOT NULL,
   price      NUMBER (10, 2) NOT NULL
)
LOGGING;

ALTER TABLE showtimes ADD CONSTRAINT showtimes_pk PRIMARY KEY (show_id);

CREATE TABLE theatercityhall (
   theater_id   NUMBER
      GENERATED ALWAYS AS IDENTITY (START WITH 1 NOCACHE)
   NOT NULL,
   theater_name VARCHAR2(100) NOT NULL,
   location     VARCHAR2(200),
   capacity     NUMBER NOT NULL
)
LOGGING;
```

```
ALTER TABLE theatercityhall ADD CONSTRAINT theatercityhall_pk PRIMARY KEY (theater_id);

CREATE TABLE ticket (
   ticket_id    NUMBER
      GENERATED ALWAYS AS IDENTITY (START WITH 1 NOCACHE)
   NOT NULL,
   user_id      NUMBER,
   show_id      NUMBER,
   booking_date DATE DEFAULT sysdate,
   seat_number VARCHAR2(10),
   status       VARCHAR2(20) DEFAULT 'Paid'
)
LOGGING;

ALTER TABLE ticket ADD CONSTRAINT ticket_pk PRIMARY KEY (ticket_id);

CREATE TABLE users (
   user_id       NUMBER
      GENERATED ALWAYS AS IDENTITY (START WITH 1 NOCACHE)
   NOT NULL,
   username      VARCHAR2(50) NOT NULL,
   user_password VARCHAR2(100) NOT NULL,
   email         VARCHAR2(100),
   phone_number VARCHAR2(20),
   address       VARCHAR2(200)
)
LOGGING;

ALTER TABLE users ADD CONSTRAINT users_pk PRIMARY KEY (user_id);

ALTER TABLE users ADD CONSTRAINT index_1 UNIQUE (email);

ALTER TABLE showtimes
   ADD CONSTRAINT showtimes_movie_fk
      FOREIGN KEY (movie_id)
```

```
        REFERENCES movie (movie_id)
            ON DELETE CASCADE
        NOT DEFERRABLE;

ALTER TABLE showtimes
    ADD CONSTRAINT showtimes_theatercityhall_fk
        FOREIGN KEY (theater_id)
        REFERENCES theatercityhall (theater_id)
            ON DELETE CASCADE
        NOT DEFERRABLE;

ALTER TABLE ticket
    ADD CONSTRAINT ticket_showtimes_fk
        FOREIGN KEY (show_id)
        REFERENCES showtimes (show_id)
            ON DELETE CASCADE
        NOT DEFERRABLE;

ALTER TABLE ticket
    ADD CONSTRAINT ticket_users_fk
        FOREIGN KEY (user_id)
        REFERENCES users (user_id)
            ON DELETE CASCADE
        NOT DEFERRABLE;
```

```
Script Output
Task completed in 3.109 seconds
Table MOVIE created.

Table MOVIE altered.

Table THEATERCITYHALL created.

Table THEATERCITYHALL altered.

Table USERS created.

Table USERS altered.

Table USERS altered.
```

Figure 3 output

```
Script Output
Task completed in 3.109 seconds
Table SHOWTIMES created.

Table SHOWTIMES altered.

Table TICKET created.

Table TICKET altered.

Table SHOWTIMES altered.

Table SHOWTIMES altered.

Table TICKET altered.
```

Figure 4 output

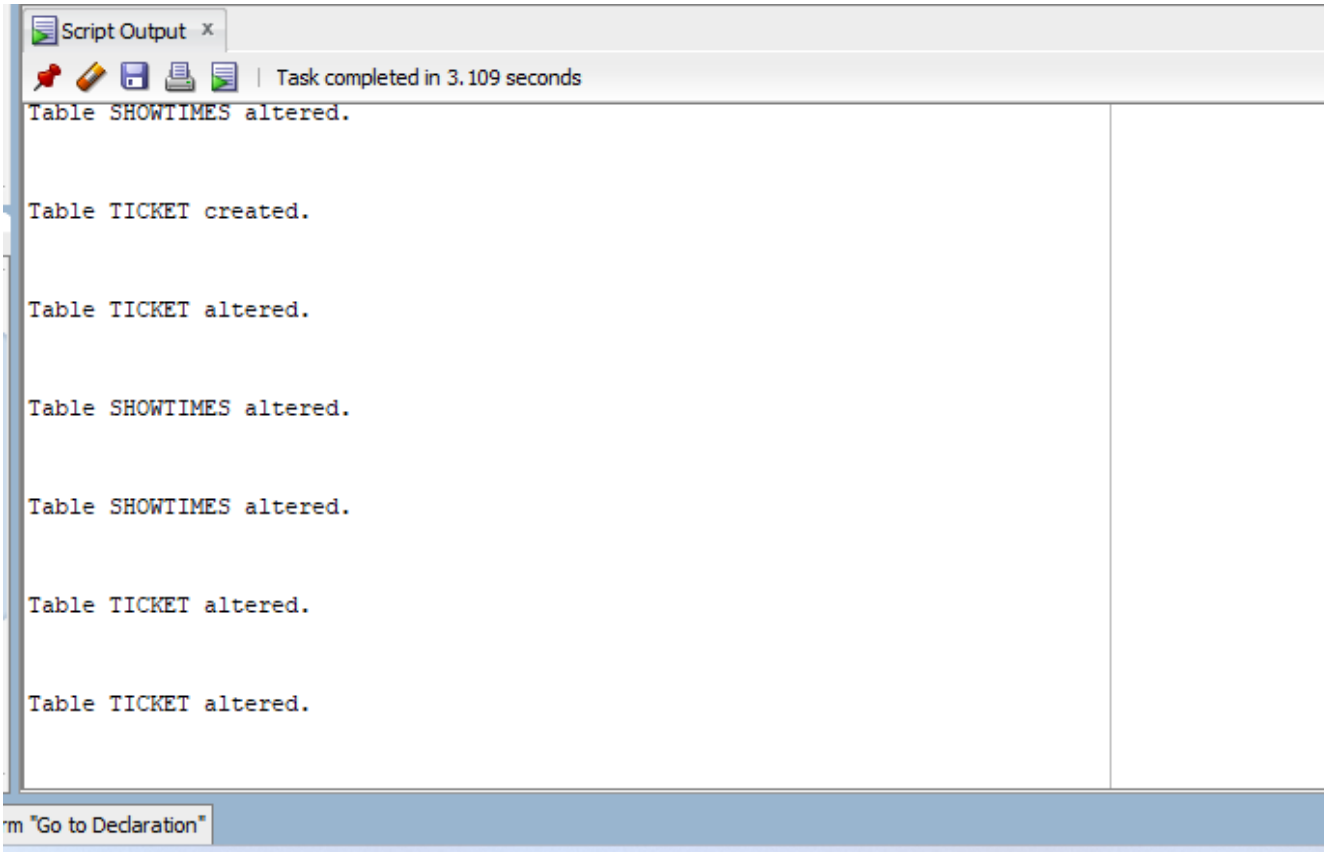

Figure 5 output

9 Insert Statement

Worksheet | Query Builder

```
INSERT INTO USERS (USERNAME, USER_PASSWORD, EMAIL, PHONE_NUMBER, ADDRESS)
VALUES ('admin_user', 'Admin@123', 'admin@kumari.com', '9801234567', 'Durbarmarg, Kathmandu');
INSERT INTO USERS (USERNAME, USER_PASSWORD, EMAIL, PHONE_NUMBER, ADDRESS)
VALUES ('puja_chaudhary', 'Puja@2024', 'puja.c@example.com', '9812345678', 'Lalitpur, Nepal');
INSERT INTO USERS (USERNAME, USER_PASSWORD, EMAIL, PHONE_NUMBER, ADDRESS)
VALUES ('ram_shrestha', 'Ram@Pass', 'ram.s@test.com', '9841112223', 'Bhaktapur, Nepal');

INSERT INTO THEATERCITYHALL (THEATER_NAME, LOCATION, CAPACITY)
VALUES ('Kumari Hall 1', 'Kathmandu', 150);
INSERT INTO THEATERCITYHALL (THEATER_NAME, LOCATION, CAPACITY)
VALUES ('Kumari Hall 2 (Ultra)', 'Kathmandu', 100);


INSERT INTO MOVIE (MOVIE_TITLE, GENRE, LANGUAGE, DURATION)
VALUES ('Avatar: The Way of Water', 'Action/Sci-Fi', 'English', 192);
INSERT INTO MOVIE (MOVIE_TITLE, GENRE, LANGUAGE, DURATION)
VALUES ('Purnabir', 'Drama', 'Nepali', 145);
INSERT INTO MOVIE (MOVIE_TITLE, GENRE, LANGUAGE, DURATION)
VALUES ('Oppenheimer', 'Biography', 'English', 180
INSERT INTO SHOWTIMES (MOVIE_ID, THEATER_ID, SHOW_TIME, PRICE)
VALUES (1, 1, TO_TIMESTAMP('2026-03-25 14:00:00', 'YYYY-MM-DD HH24:MI:SS'), 550.00);

INSERT INTO SHOWTIMES (MOVIE_ID, THEATER_ID, SHOW_TIME, PRICE)
VALUES (2, 2, TO_TIMESTAMP('2026-03-25 18:00:00', 'YYYY-MM-DD HH24:MI:SS'), 450.00);
```

Figure 6 Insert query

Worksheet | Query Builder

```
VALUES ('Purnabir', 'Drama', 'Nepali', 145);
INSERT INTO MOVIE (MOVIE_TITLE, GENRE, LANGUAGE, DURATION)
VALUES ('Oppenheimer', 'Biography', 'English', 180
INSERT INTO SHOWTIMES (MOVIE_ID, THEATER_ID, SHOW_TIME, PRICE)
VALUES (1, 1, TO_TIMESTAMP('2026-03-25 14:00:00', 'YYYY-MM-DD HH24:MI:SS'), 550.00);

INSERT INTO SHOWTIMES (MOVIE_ID, THEATER_ID, SHOW_TIME, PRICE)
VALUES (2, 2, TO_TIMESTAMP('2026-03-25 18:00:00', 'YYYY-MM-DD HH24:MI:SS'), 450.00);

INSERT INTO SHOWTIMES (MOVIE_ID, THEATER_ID, SHOW_TIME, PRICE)
VALUES (3, 1, TO_TIMESTAMP('2026-03-26 10:00:00', 'YYYY-MM-DD HH24:MI:SS'), 600.00);


INSERT INTO TICKET (USER_ID, SHOW_ID, SEAT_NUMBER, STATUS)
VALUES (2, 1, 'A12', 'Paid');

INSERT INTO TICKET (USER_ID, SHOW_ID, SEAT_NUMBER, STATUS)
VALUES (3, 2, 'B05', 'Paid');

INSERT INTO TICKET (USER_ID, SHOW_ID, SEAT_NUMBER, STATUS)
VALUES (1, 3, 'F01', 'Paid');

COMMIT;
```

Figure 7 Insert Query

```
1 row inserted.

1 row inserted.

1 row inserted.

1 row inserted.

1 row inserted.
```

Figure 8 output

10 Select Statement

```
SELECT T.TICKET_ID, M.MOVIE_TITLE, S.SHOW_TIME, T.SEAT_NUMBER, T.STATUS
FROM TICKET T
JOIN SHOWTIMES S ON T.SHOW_ID = S.SHOW_ID
JOIN MOVIE M ON S.MOVIE_ID = M.MOVIE_ID
WHERE T.USER_ID = 2;


SELECT (COUNT(T.TICKET_ID) / TH.CAPACITY) * 100 AS OccupancyRate
FROM SHOWTIMES S
JOIN THEATERCITYHALL TH ON S.THEATER_ID = TH.THEATER_ID
LEFT JOIN TICKET T ON S.SHOW_ID = T.SHOW_ID
WHERE S.SHOW_ID = 1
GROUP BY TH.CAPACITY;
```

Figure 9 selecting query

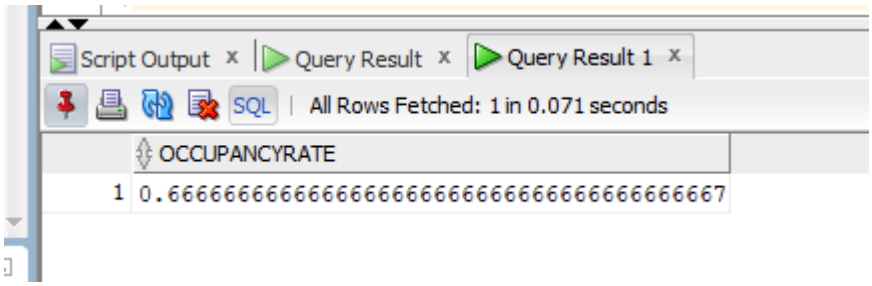

Figure 10 output

11 Forms

11.1 Dashboard Page

It is the primary landing page for administrators. It features real-time statistics (Live Intelligence) and graphical trends of booking activity.

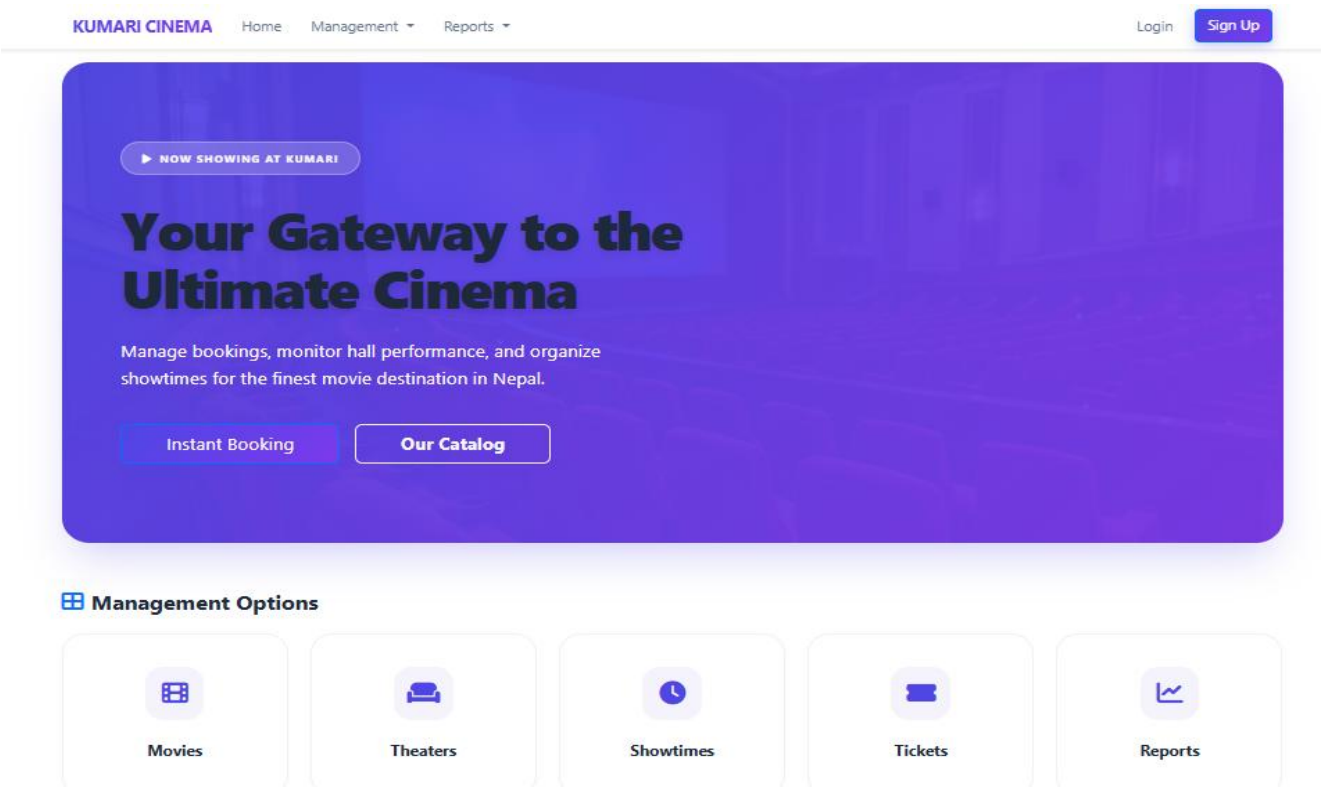

Figure 11 dashboard page

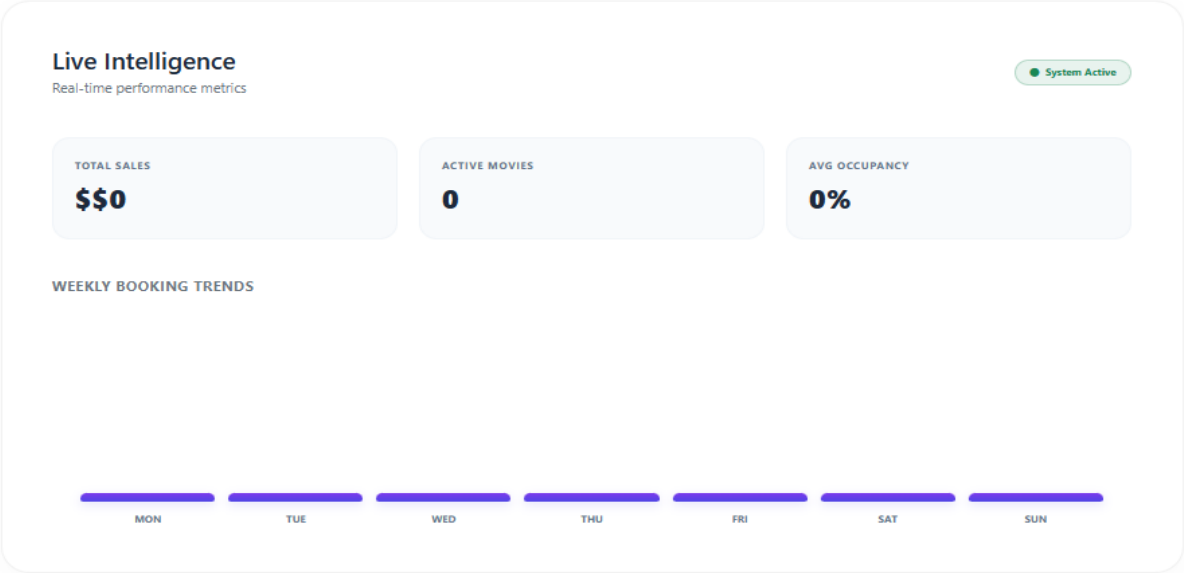

Figure 12 dashboard page

11.2 Complex Forms and Queries

1. Forms

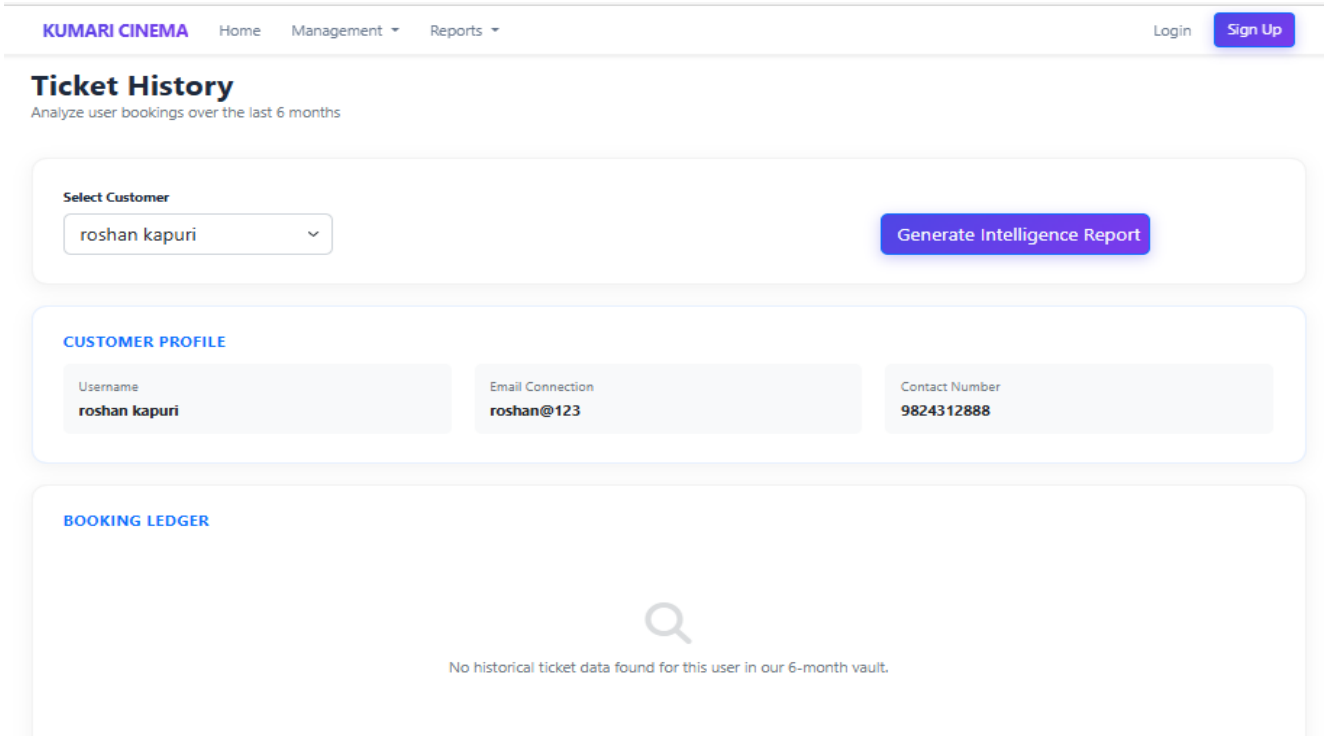

Figure 13 Ticket History Page

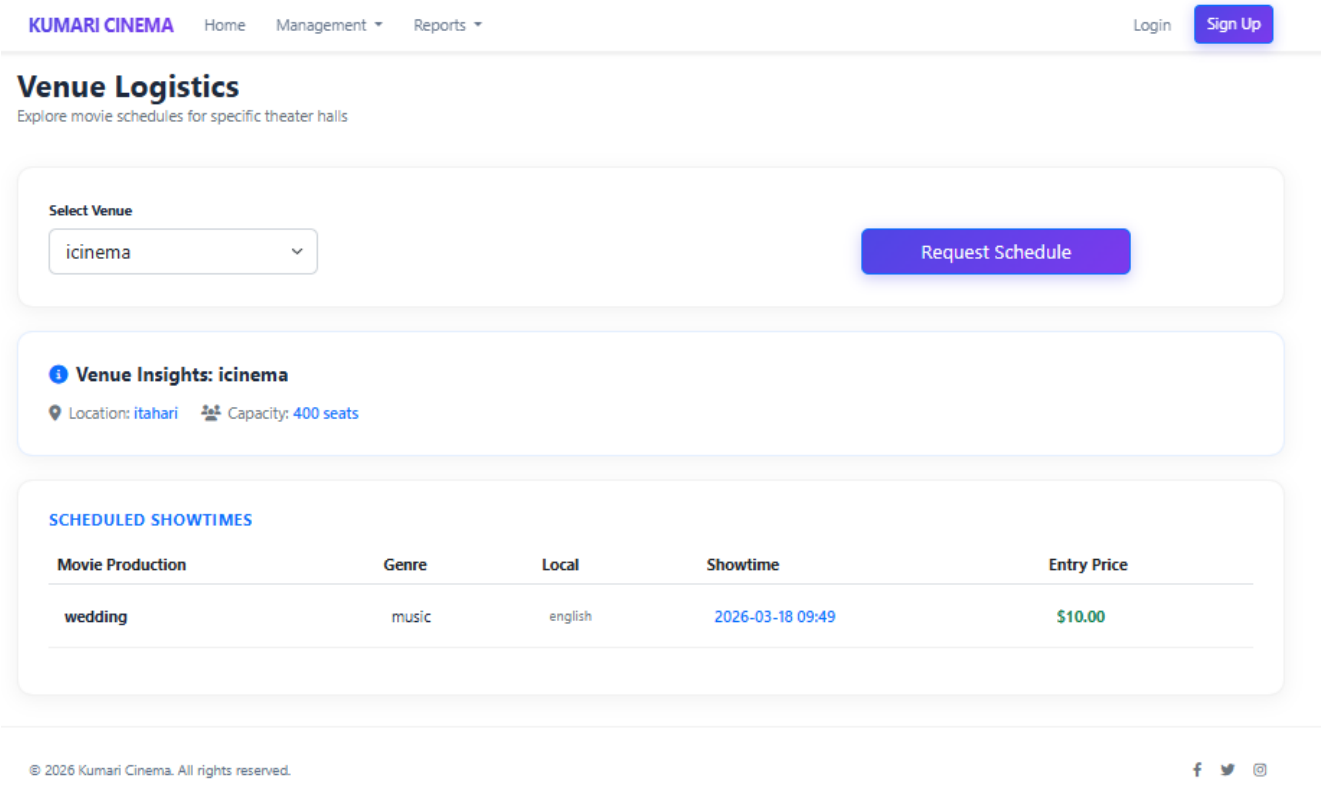

Figure 14 Venue page

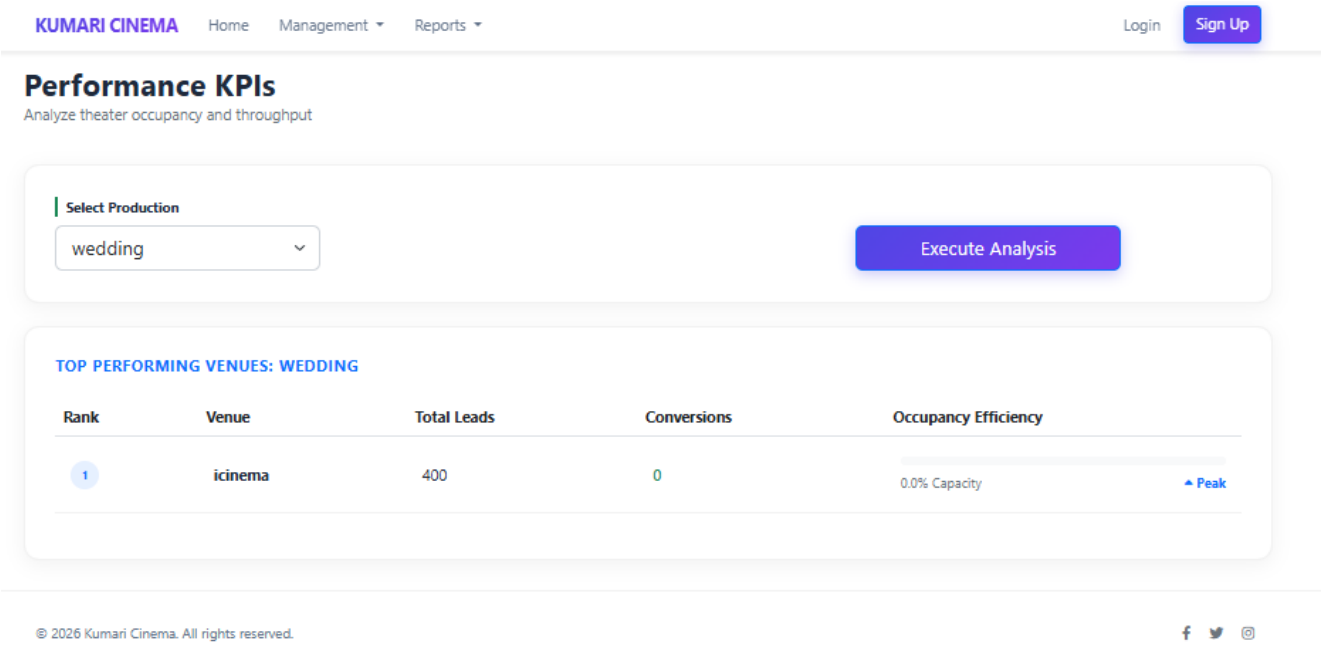

Figure 15 Performance List

2. Queries
   - User Ticket History

```
SELECT tk. TICKET_ID, m. MOVIE_TITLE, th. THEATER_NAME,

    TO_CHAR (s. SHOW_TIME, 'YYYY-MM-DD HH24:MI') as SHOW_TIME,

    tk. SEAT_NUMBER, tk. BOOKING_DATE, tk. STATUS

FROM TICKET tk

JOIN SHOWTIMES s ON tk. SHOW_ID = s. SHOW_ID

JOIN MOVIE m ON s. MOVIE_ID = M. MOVIE_ID

JOIN THEATERCITYHALL th ON s. THEATER_ID = th. THEATER_ID

WHERE tk.USER_ID =: UserId AND tk. BOOKING_DATE >= ADD_MONTHS (SYSDATE, -6)

ORDER BY tk. BOOKING_DATE DESC;
```

   - Theater Hall Movie Report

```
SELECT M. MOVIE_TITLE, m. GENRE, m. LANGUAGE,

    TO_CHAR (s. SHOW_TIME, 'YYYY-MM-DD HH24:MI') as SHOW_TIME, s. PRICE

FROM SHOWTIMES s

JOIN MOVIE m ON s. MOVIE_ID = m. MOVIE_ID
```

```
WHERE s. THEATER_ID =: Id

ORDER BY S. SHOW_TIME ASC;
```

- Occupancy Performance

```
SELECT * FROM (

  SELECT th. THEATER_NAME, th. CAPACITY, COUNT (tk. TICKET_ID) as PAID_TICKETS,

  CASE

    WHEN th. CAPACITY > 0 THEN (COUNT (tk. TICKET_ID) / (th. CAPACITY * (SELECT COUNT (*) FROM SHOWTIMES s2 WHERE s2. THEATER_ID = th. THEATER_ID AND s2. MOVIE_ID =: MovieId))) * 100

    ELSE 0

  END as OCCUPANCY_PERCENT

  FROM THEATERCITYHALL th

  JOIN SHOWTIMES s ON th. THEATER_ID = s. THEATER_ID

  LEFT JOIN TICKET tk ON s. SHOW_ID = tk. SHOW_ID AND tk. STATUS = 'Paid'

  WHERE s. MOVIE_ID =: MovieId

  GROUP BY th. THEATER_NAME, th. CAPACITY, th. THEATER_ID
```

```
    ORDER BY OCCUPANCY_PERCENT DESC

) WHERE ROWNUM <= 3;
```

11.3 Simple Forms

Simple forms in the Kumari Cinemas system provide basic CRUD (Create, Read, Update, Delete) functionality for maintaining core database entities.

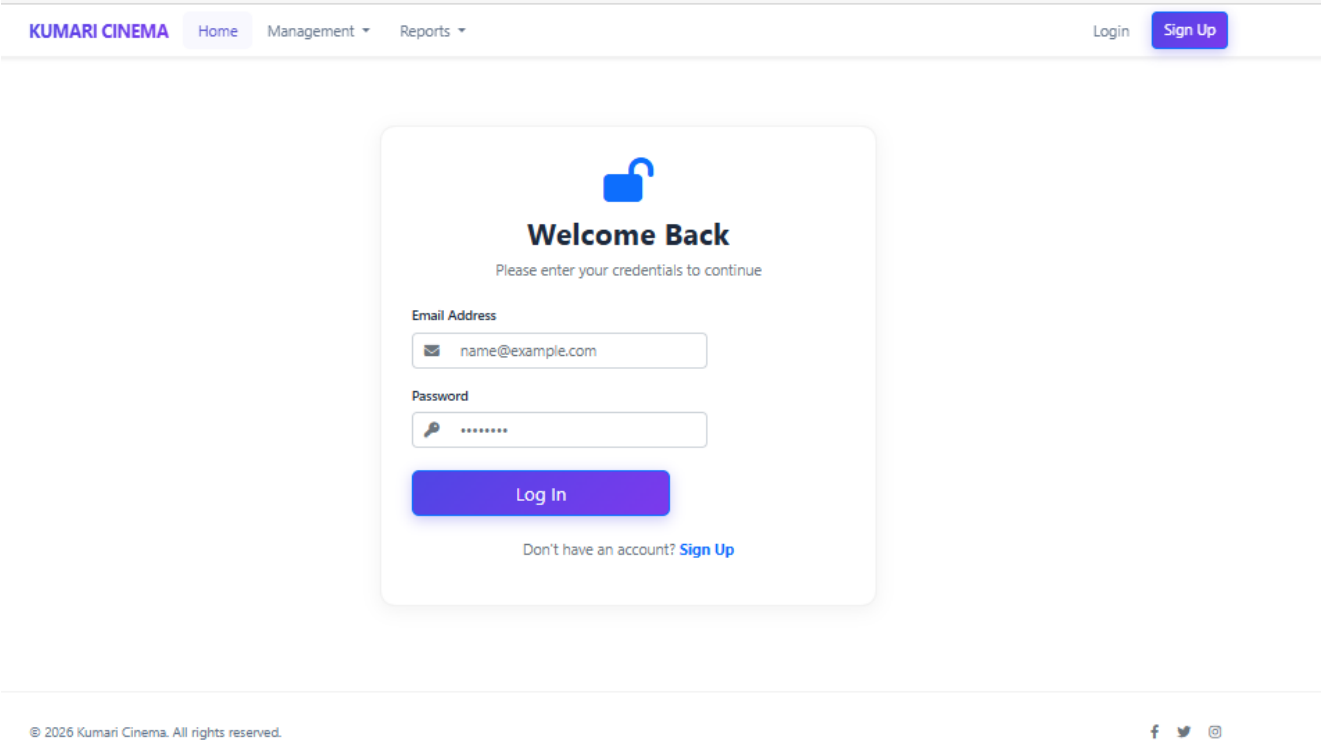

Figure 16 login page

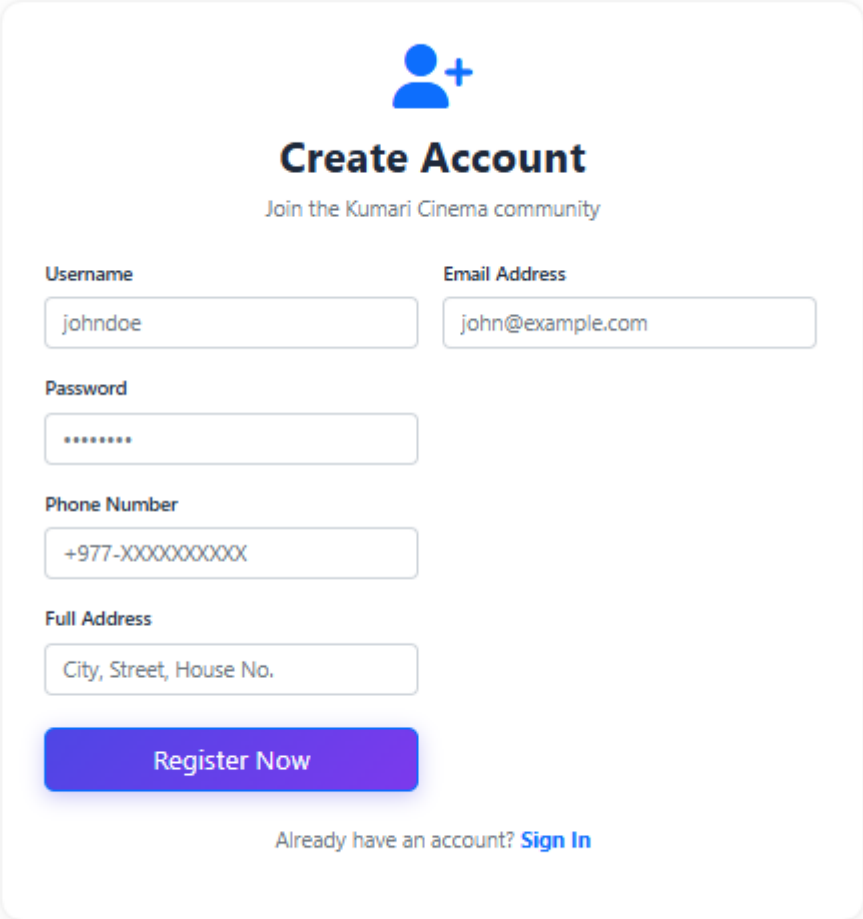

Figure 17 sign up page

12 User Manual

1. For Administrator
   - System Entry: Go to /Login.aspx and use administrator credentials.

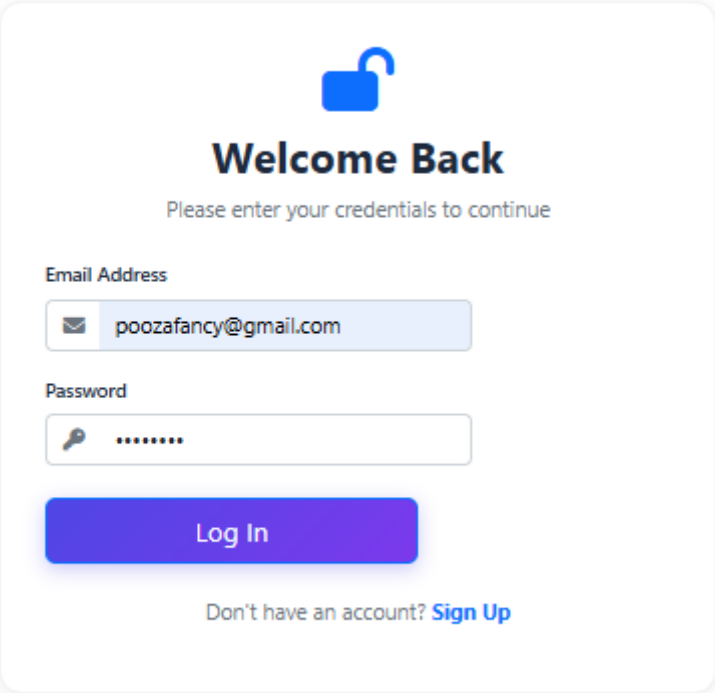

Figure 18 login page for administrator

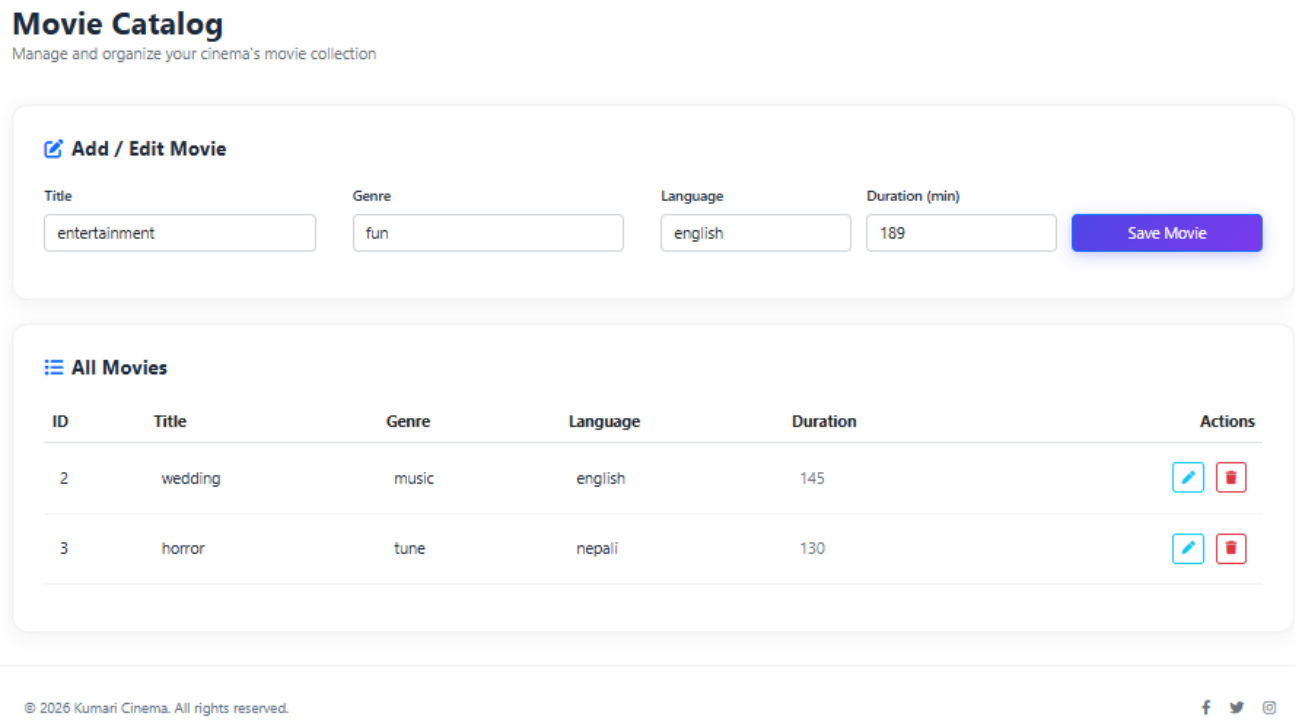

Figure 19 movie list

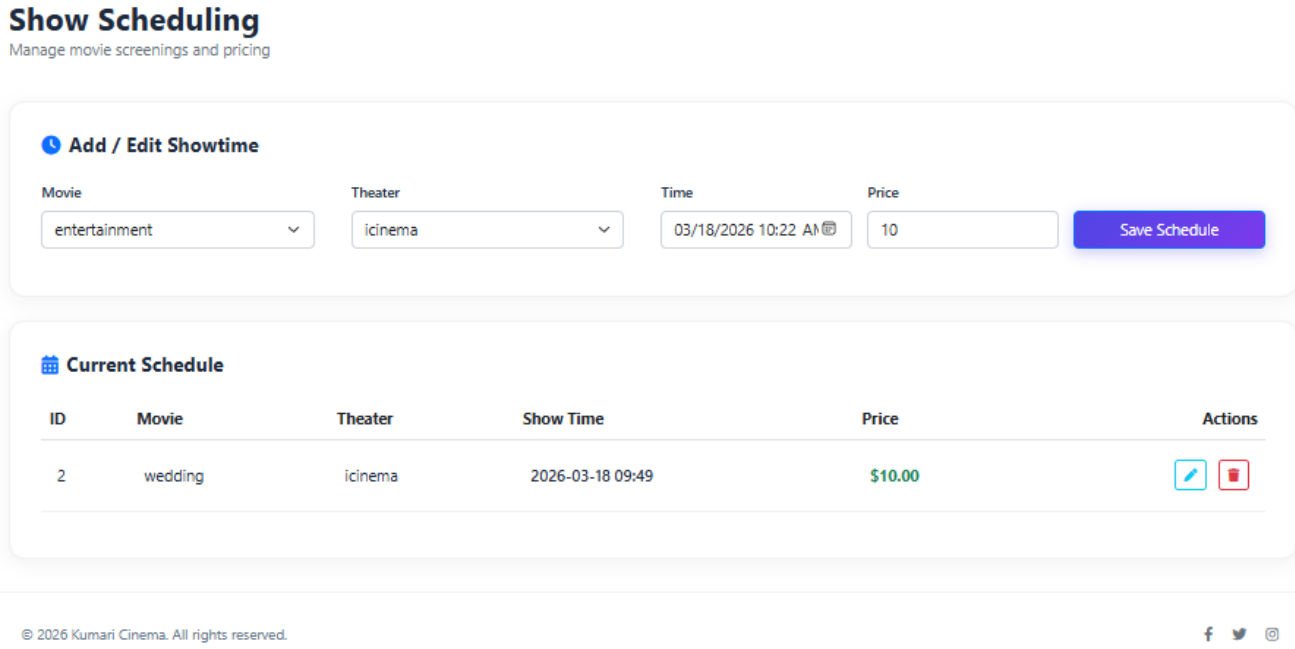

Figure 20 show list

2. For Customer

- Creation of an account: Register at /Register.aspx by entering contact information.

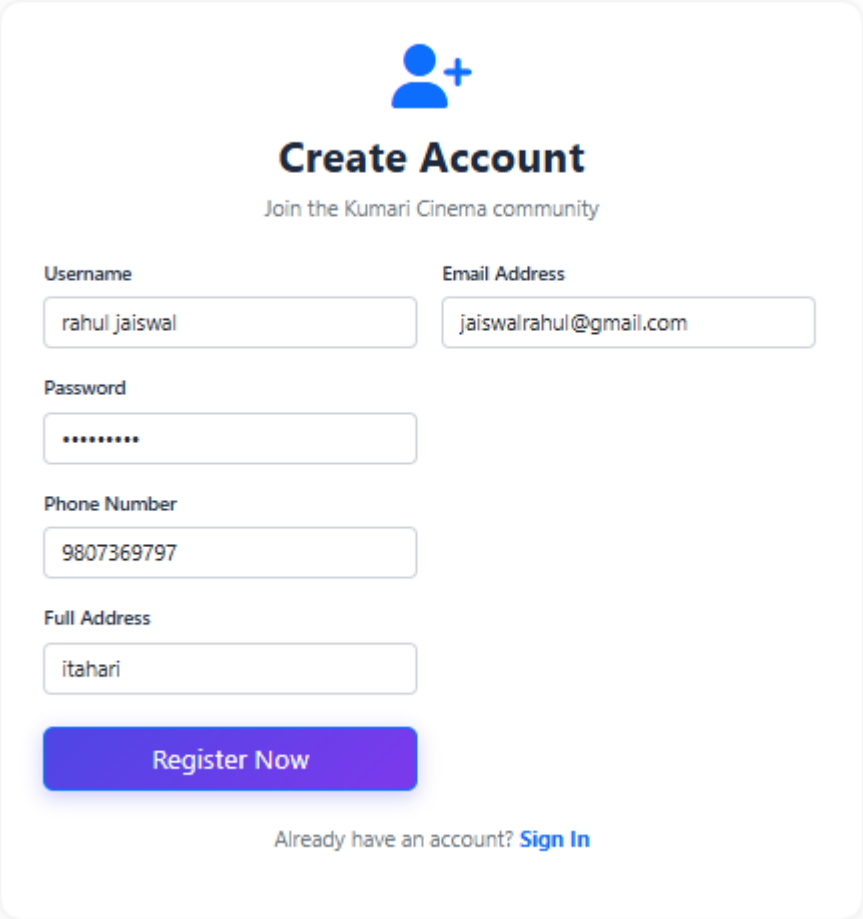

Figure 21 sign up page for customer

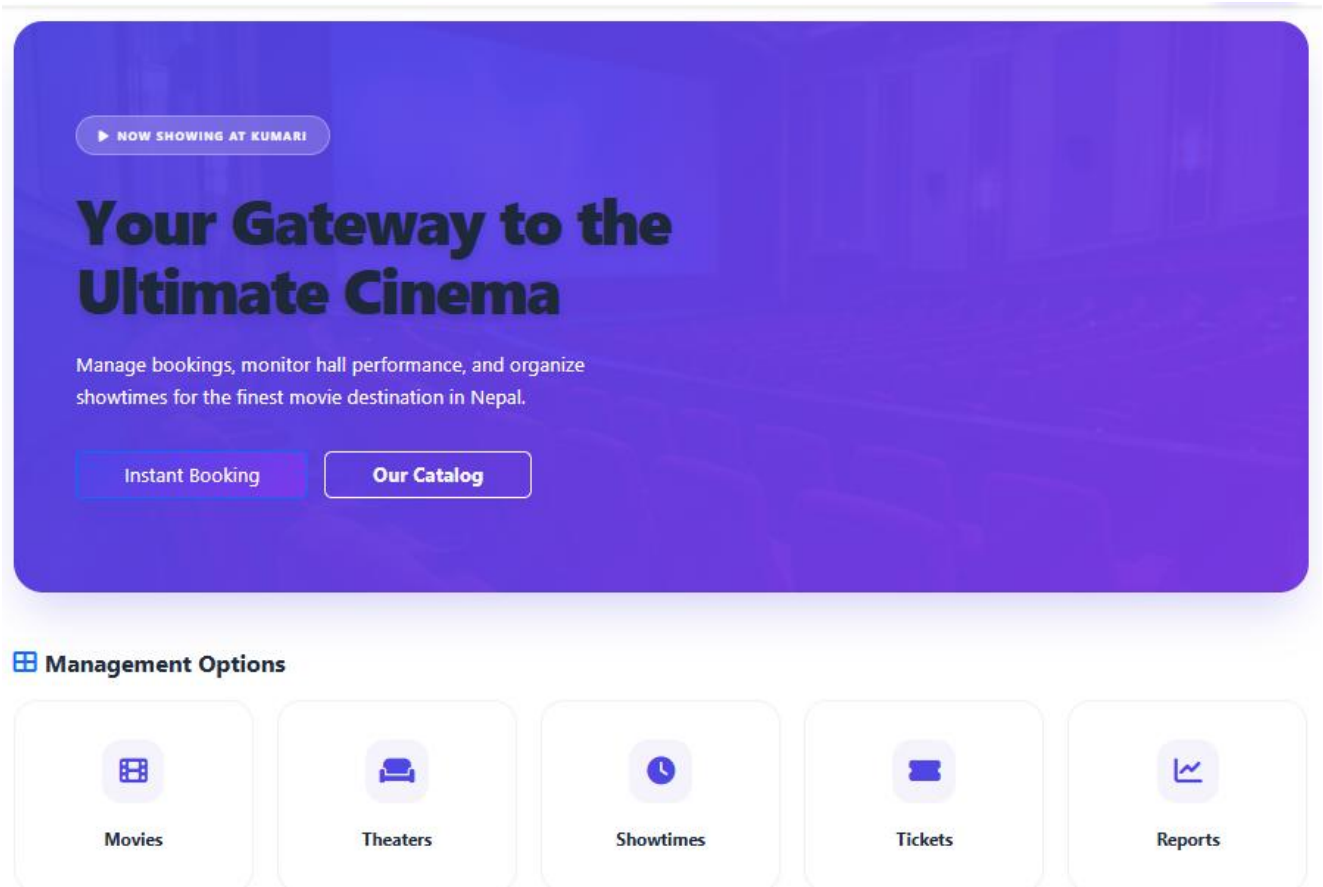

Figure 22dashboard page

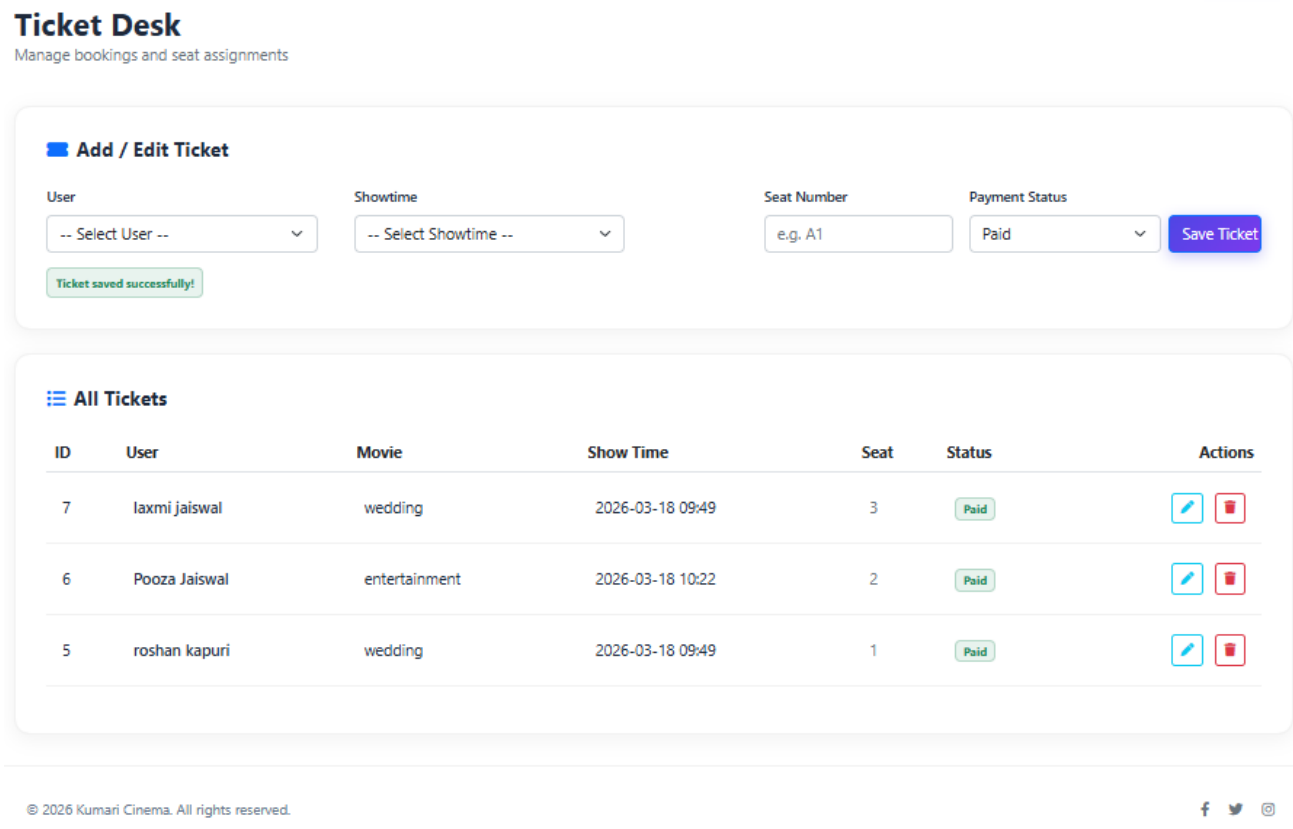

Figure 23 ticket list

13 Testing

1. Unit Testing

- User Authentication: The authentication was performed and it was clear that the 'Login.aspx' page identifies valid users accurately and successfully denies unauthorized access.

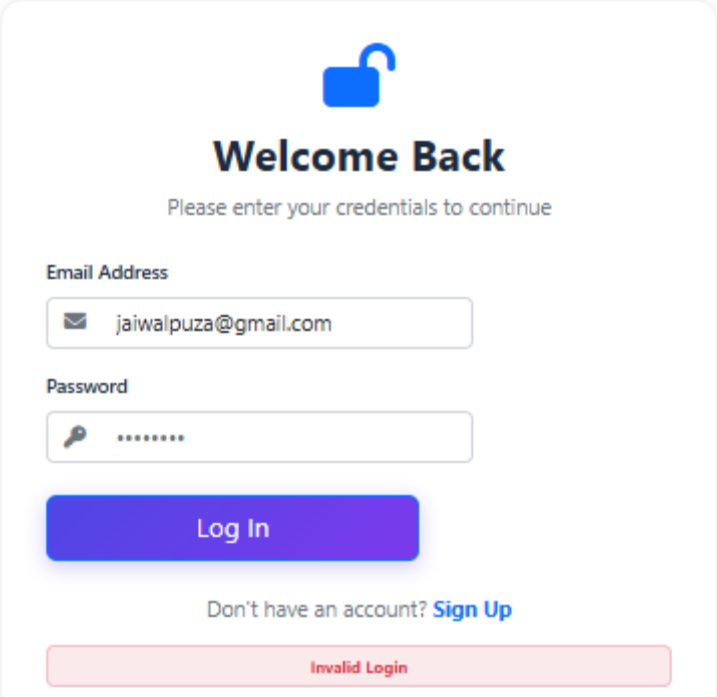

Figure 24 check if invalid email reject

- Data Check: It was ensured that theater capacity, ticket prices and showtime durations have been provided as positive numerical values.

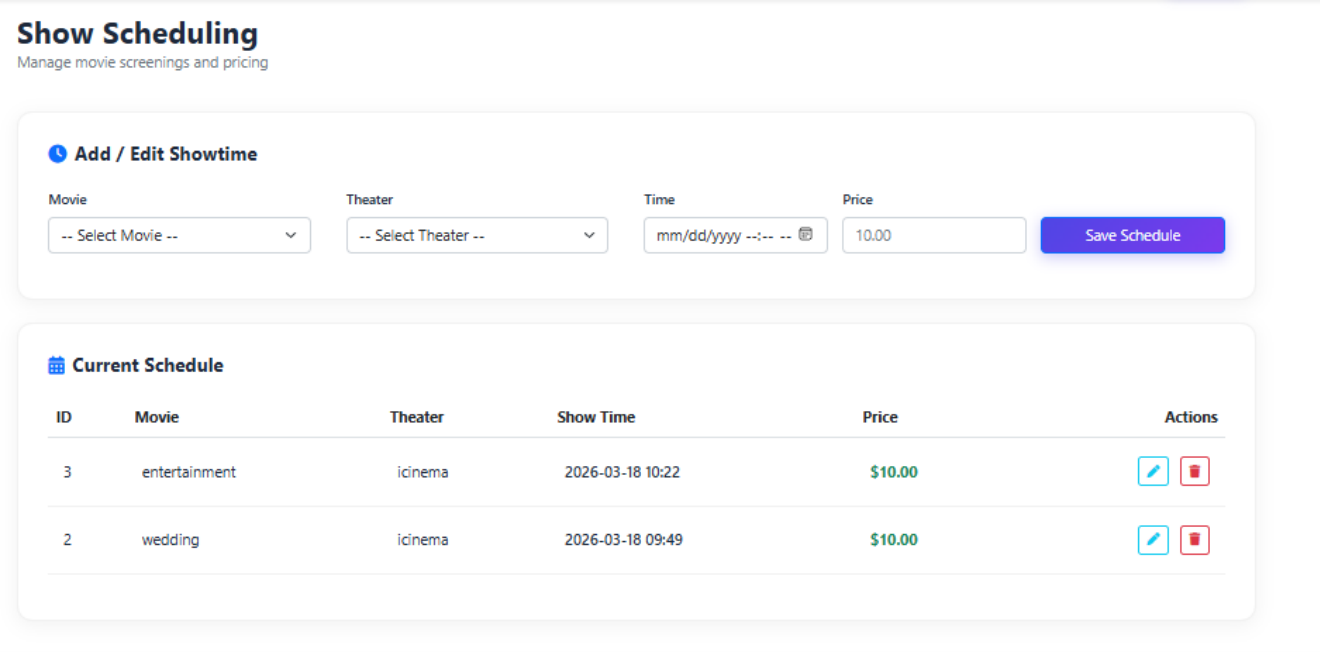

Figure 25 show list

- Integrity Cascading: It was confirmed that a removal of a record in the MOVIE and all related records in SHOWTIMES and TICKET have been deleted automatically, thus preventing the existence of orphaned records.

2. Integration Testing

- End-to-End Booking: The entire process starts with registering a user into a platform, browsing through the movie catalog and book a seat and concludes with the ensuring that the booking is reflected in the user history was strictly tested.
- Constraint Enforcement: Verification was done to make sure the database did not accept duplicate seat booking of the same showtime, thus avoiding cases of a showtime being double sold.
- Report Accuracy: Occupancy calculations using Coded occupancy were compared to the output of OccupancyReport.aspx to ensure that the SQL logic is 100 per cent correct.

Test Cases

TestID	Feature	Input/Action	ExpectedResult	Status
TC-01	User Signup	Valid email and password	Record added to users table, redirected to login	Pass
TC-02	Movie Admin	Add drama	Movie appears in management grid and booking dropdown	Pass
TC-03	Seat Conflict	Book seat A1 twice	System throws an error or hides the seat after first booking	Pass
TC-04	Occupancy	75 tickets sold in 100-seat hall	Report displays 75. oo% occupancy rate	Pass
TC-05	History Filter	View tickets for admin	Only tickets belonging to user_id=1 are displayed in the grid	Pass

Table 8 Test Cases

14 Further Discussion

Although the existing system is consistent with all essential requirements, a number of usability variants for future improvement have been outlined:

- Pay Gateway Cooperation: Direct interconnection with the service like eSewa or Khalti to accommodate Nepalese users.
- Mobile Application: A specific Android/iOS application will be developed using the existing database back end.
- Multi-Language Support: Interface in both the English and Nepali language.
- QR Code Tickets: Creation of exclusive QR code for tickets in order to facilitate the confirmation of entry at the cinemas.

15 Conclusion

The Kumari Cinemas Management System is a good way to provide the Nepali cinema industry with a digital solution. Through the application of an Oracle database that has been normalized and application of the latest ASP.net front-end, the system ensures integrity of the data as well as providing excellent user experience. The project is a good demonstration case of how database design principles, CRUD operations, and real-time performance reporting can be successfully used.

The database has been normalized step by step by step by using a systematic refinement procedure, starting with Unnormalized Form (UNF) to the Third Normal Form (3NF). The process has reduced redundancy, maintained data integrity, and ensured effective querying. Basic objects were User, Movie, Theatre, Hall, Show, and Ticket which were carefully designed and modeled in a primary and foreign key relationship which produced a maintainable and sensible schema.

The completed Entity-Relationship Diagram (ERD) and the Data Dictionary provide a detailed list of the relationships within the system, data types, data constraints, and the relationships between the tables within the system. The next SQL implementation depicted the real-life construction and population of tables with sample data, which confirmed that the database was operational enough to be deployed in the real world.

This database gives a solid base to the future automated ticketing system of Kumari Cinemas, which is expected to improve its operational effectiveness, customer satisfaction and allow it to grow its operations in other locations by necessity. This milestone has been accomplished, which shows an excellent mastery of the concepts of database design and how they can be applied to the complex business issues.

16 References

Connolly, 2015. Database Systems: A Practical Approach to Design, Implementation, and Management. Harlow: Pearson.

Elmasri, 2016. *Fundamentals of Database Systems.* Boston: Pearson.

Oracle Corporation, 2023. *What is a Database?.* [Online] Available at: https://www.oracle.com/database/what-is-database/ [Accessed 18 March 2026].

Statista, 2023. *Cinema industry digital transformation and ticketing systems.* [Online] Available at: https://www.statista.com/topics/964/film/ [Accessed 18 March 2026].